

Reviewer Pack — Tree-to-Dimension Tightness for MAJ_3: Optimal KW Protocols ...

Entry 792c33fb69ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 06:34:25 UTC

Tree-to-Dimension Tightness for MAJ_3: Optimal KW Protocols Yield Multiplicity-4 Covers of $X_{\{\text{MAJ}_3\}}$

Entry ID: 792c33fb69ec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 06:34:25 UTC

1. Conjecture

Field A (mathematical branch): Coarse-Geometric KW Bounds (CG-KW) framework; coarse geometry and asymptotic dimension (Yu/Nowak-Yu) applied to Karchmer-Wigderson games **Field B** (complexity object): De Morgan formula depth lower bounds in the Karchmer-Wigderson communication model, specifically for the monotone threshold function MAJ_3 whose known optimal depth $D(\text{MAJ}_3) = 3$

Statement:

Let $X_{\{\text{MAJ}_3\}} = \text{MAJ}_3^{\{-1\}}(0) \times \text{MAJ}_3^{\{-1\}}(1)$ be the KW metric space of the 3-bit majority function under the disagreement-Hamming metric $\bar{d}((x,y),(x',y')) = \text{Hamming}(x,x') + \text{Hamming}(y,y')$. Under axiom A1 (Tree-to-Dimension), the optimal depth-3 KW Coarse_partition_tree for MAJ_3 induces, via tree_to_cover, an AsymDim_cover $\{U_a\}$ of $X_{\{\text{MAJ}_3\}}$ whose r -multiplicity is at most $4 = D(\text{MAJ}_3) + 1$ for every $r \geq 1$, with each U_a having diameter ≤ 4 . Equivalently, $\text{asdim_at_scale}(X_{\{\text{MAJ}_3\}}) \leq D(\text{MAJ}_3)$, and conversely no cover of multiplicity ≤ 3 achieves diameter ≤ 2 , witnessing the depth lower bound $\text{asdim}(X_{\{\text{MAJ}_3\}}) \geq 2$ predicted by the framework's target invariant formula_depth_D(f).

Rationale (proposer's reasoning):

This sub-conjecture stress-tests axiom A1 on the smallest non-trivial monotone threshold function whose optimal formula depth is exactly known ($D(\text{MAJ}_3)=3$, attained by the standard balanced AC^0 formula). If A1 is correct, an explicit depth-3 KW protocol must yield a cover with multiplicity bounded by 4 and small diameter; conversely the converse one-sided gap (no multiplicity-3 cover at small scale) certifies that the framework's coarse invariant captures genuine depth lower bounds rather than collapsing trivially. MAJ_3 also serves as a non-PARITY witness: PARITY's KW geometry is highly symmetric (a metric quotient of the cube), whereas MAJ_3 has nontrivial threshold structure, so confirming A1 here generalizes the existing PARITY sub-conjecture to a genuinely different combinatorial regime.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_6ef88ddbf3, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cdc5285551689fb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 5 deterministic seeds (S_3 coordinate permutations) of the canonical depth-3 KW protocol for MAJ₃, the induced leaf cover must have r -multiplicity ≤ 4 and per-leaf diameter ≤ 4 for all $r \in \{1, 2, 3\}$; AND the brute-force search must confirm no cover of $X_{\{\text{MAJ}_3\}}$ with multiplicity ≤ 3 achieves diameter ≤ 2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.88	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer Wigderson asymptotic dimension formula depth lower bound - coarse geometry communication complexity KW game majority - asymptotic dimension cover multiplicity Karchmer-Wigderson MAJ

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from collections import defaultdict

def popcount(n):
    count = 0
    while n:
        count += n & 1
        n >>= 1
    return count

def hamming(x, y):
    return popcount(int(x, 2) ^ int(y, 2))

def distance_matrix(X):
    n = len(X)
    D = [[0] * n for _ in range(n)]
    for i in range(n):
```

```

        for j in range(i + 1, n):
            D[i][j] = hamming(X[i], X[j])
            D[j][i] = D[i][j]
    return D

def tree_to_cover(tree, leaves, r):
    cover = defaultdict(list)
    queue = [(node, r) for node in leaves if tree[node]['depth'] == 0]
    while queue:
        node, r = queue.pop(0)
        cover[r].append(node)
        for child in tree[node]['children']:
            queue.append((child, r - 1))
    return cover

def run_trial(seed: int) -> dict:
    random.seed(seed)

    X = ['000', '001', '010', '100', '011', '101', '110', '111']
    D = distance_matrix(X)

    # Construct the depth-3 KW protocol tree
    leaves = list(range(len(X)))
    tree = {i: {'depth': 0, 'children': []} for i in range(len(X))}
    for i in range(4):
        new_leaves = []
        for leaf in leaves:
            x = X[leaf]
            if i == 0:
                child1 = len(tree)
                tree[child1] = {'depth': i + 1, 'children': []}
                tree[leaf]['children'].append(child1)
                new_leaves.append(child1)
                child2 = len(tree)
                tree[child2] = {'depth': i + 1, 'children': []}
                tree[leaf]['children'].append(child2)
                new_leaves.append(child2)
            else:
                child = len(tree)
                tree[child] = {'depth': i + 1, 'children': []}
                tree[leaf]['children'].append(child)
                new_leaves.append(child)
        leaves = new_leaves

    # Run tree_to_cover for r = 1, 2, 3
    covers = [tree_to_cover(tree, leaves, r) for r in range(1, 4)]

    # Check multiplicity and diameter
    multiplicity = max(len(covers[r][r]) for r in range(1, 4))
    diameter = max(max(D[leaf1][leaf2] for leaf2 in covers[r][r]) for r in range(1, 4))

    # Brute-force search for covers of size <= 8 with set diameter <= 2
    def brute_force():
        from itertools import combinations
        for cover_size in range(1, 9):
            for cover in combinations(range(len(X)), cover_size):

```

```

        if all(D[leaf1][leaf2] <= 2 for leaf1, leaf2 in combinations(cover, 2)):
            multiplicity = max(multiplicity for leaf in cover)
    return multiplicity

brute_multiplicity = brute_force()

# Determine if the conjecture holds
conjecture_holds = multiplicity <= 4 and diameter <= 4 and brute_multiplicity > 3

return {
    "metric_name": "Multiplicity and Diameter",
    "metric_value": (multiplicity, diameter),
    "instances_tested": len(X),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Multiplicity {brute_multiplicity} > 3"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_multiplicity = sum(r['metric_value'][0] for r in results) / len(results)
    std_multiplicity = (sum((r['metric_value'][0] - mean_multiplicity)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_multiplicity} std={std_multiplicity} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"Multiplicity {results[first_failing_seed]['metric_value'][0]} > 4\"")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f5e247f.py", line 110, in <module>
    result = run_trial(seed)
              ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f5e247f.py", line 58, in run_trial
    x = X[leaf]
        ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` before producing any `RESULT` line, so neither the support nor the falsification condition was evaluated. Per rule 2, a crashed test yields `INCONCLUSIVE`. | next: Fix the leaf-indexing bug in `run_trial` (ensure the KW protocol's leaf enumeration matches the X array length) and rerun the 5-seed protocol plus brute-force multiplicity check.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	7163
2	novelty	claude_max	opus	0	0	4519
3	novelty	claude_max	opus	0	0	6890
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12538
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13235
6	judge	claude_max	opus	0	0	6992

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 51337 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/792c33fb69ec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/792c33fb69ec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/792c33fb69ec.tar.gz` (if generated)